Multi – level Linked List – Memory Allocation

```
Node *newNode = (Node *)malloc(sizeof(Node));
```

→ *The size of newNode will be equal to the size of a Node structure and newNode is declared as a pointer to Node structure.*

→ *The size of a pointer (newNode) is typically 8 bytes on most modern 64 – bit systems, regardless of the size of the structure it points to.*

→ *The size of the Node structure itself is determined by the sum of the sizes of its members*:

- → *data(an integer) is typically 4 bytes.*
- → *next(a pointer to another Node) is typically 8 bytes.*
- → *child(a pointer to previous Node) is typically 8 bytes.*

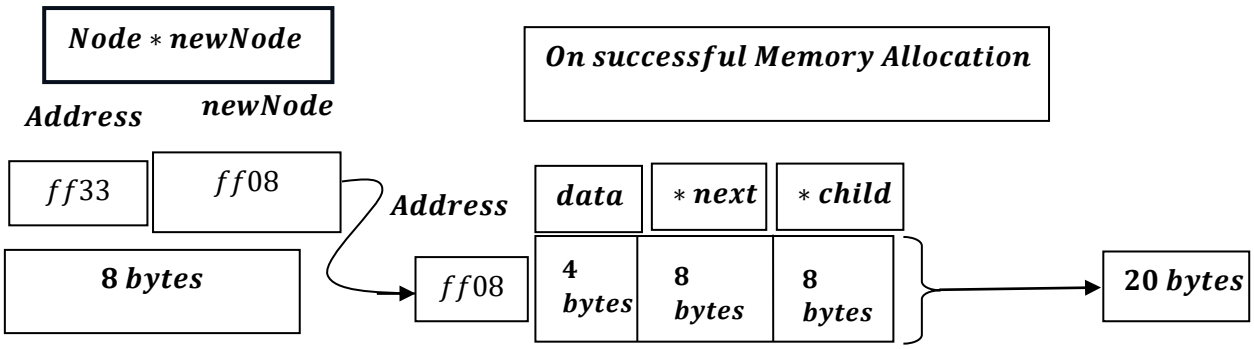
→ *Therefore , the size of the Node structure is typically 20 bytes(4 bytes for data + 8 bytes for next + 8 bytes for child) .*

```
Node *newNode = (Node *)malloc(sizeof(Node));
```

What does it mean?

→ *newNode will try to allocate memory for data i.e. 4 bytes , next pointer variable i.e. typically 8 bytes, and child pointer variable that is typically 8 bytes i.e. total 20 bytes, as discussed earlier , the size of a pointer (newNode) is typically 8 bytes on most modern 64 – bit systems, regardless of the size of the structure it points to.*

Say,



The above is Theoretical , But if we do a reality check:

Theoretical vs. Real – World Allocation

The above calculation is only theoretical.

In a real C/ C++ program, compilers align members for faster memory access.

Most 64 – bit compilers align pointers to 8 – byte boundaries.

Since:

```
    int data;
```

occupies only 4 bytes, the compiler inserts – 4 bytes of padding before the first pointer.

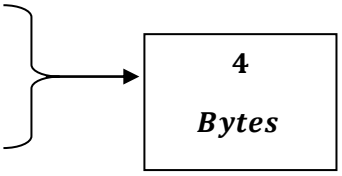
Offset	Size	Content	Description
0	4 bytes	data	Integer value
4	4 bytes	padding	Empty space (alignment)
8	8 bytes	next	Address of next node
16	8 bytes	child	Address of child node

Actually it is:

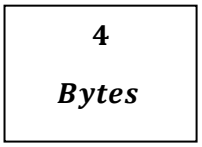
Offset	Size	Content	Description
0 – 3	4 bytes	data	Integer value
4 – 7	4 bytes	padding	Empty space (alignment)
8 – 15	8 bytes	next	Address of next node
16 – 23	8 bytes	child	Address of child node

So:

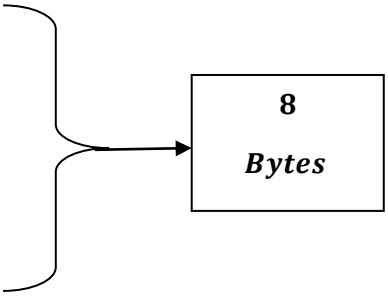
- *Offset* 0 = *first byte of the integer*(*data*)
- *Offset* 1 = *second byte of the integer*(*data*)
- *Offset* 2 = *third byte of the integer*(*data*)
- *Offset* 3 = *fourth byte of the integer*(*data*)



- *Offset* 4 = *first byte of the padding*
- *Offset* 5 = *second byte of the padding*
- *Offset* 6 = *third byte of the padding*
- *Offset* 7 = *fourth byte of the padding*

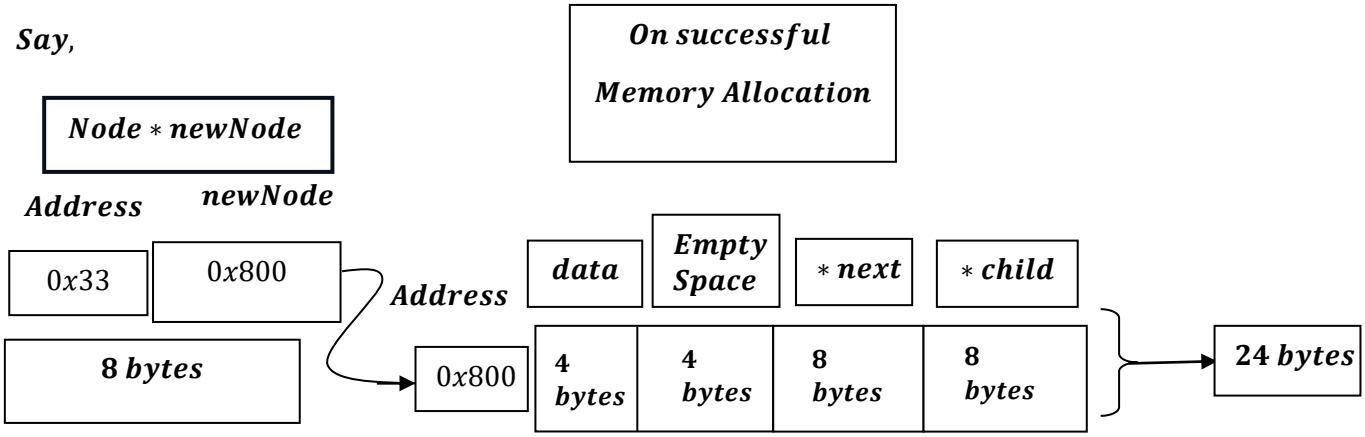


- *Offset* 8 = *first byte of the next pointer*
- *Offset* 9 = *second byte of the next pointer*
- *Offset* 10 = *third byte of the next pointer*
- *Offset* 11 = *fourth byte of the next pointer*
- *Offset* 12 = *fifth byte of the next pointer*
- *Offset* 13 = *sixth byte of the next pointer*
- *Offset* 14 = *seventh byte of the next pointer*
- *Offset* 15 = *eighth byte of the next pointer*

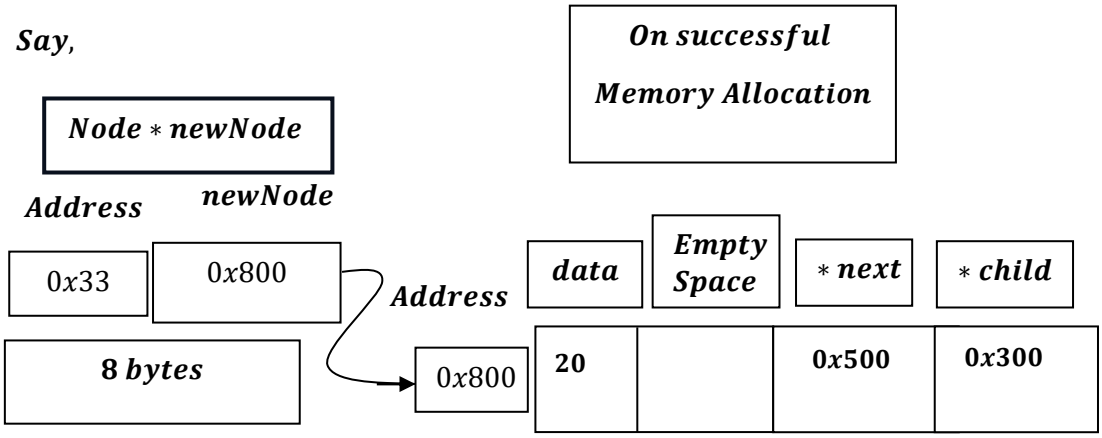


- ***Offset* 16 = *first byte of the child pointer***
- ***Offset* 17 = *second byte of the child pointer***
- ***Offset* 18 = *third byte of the child pointer***
- ***Offset* 19 = *fourth byte of the child pointer***
- ***Offset* 20 = *fifth byte of the child pointer***
- ***Offset* 21 = *sixth byte of the child pointer***
- ***Offset* 22 = *seventh byte of the child pointer***
- ***Offset* 23 = *eighth byte of the child pointer***

8 *Bytes*



***Like*:**



In a $64-$ *bit environment, the compiler aligns memory to the size of the largest primitive member in a structure* $-$

The largest primitive member in this structure is an $8-$ *byte pointer* $(child\ or\ next)$. *Therefore, the compiler aligns the entire structure according to* $8-$ *byte alignment. That means every* $8-$ *byte pointer should begin at an address that is a multiple of* 8.

Because the data integer only occupies 4 *bytes, the compiler inserts* empty *space* $(padding)$ *to ensure the pointer starts at an address that is a multiple of* 8.

Such as:

$0x500 = 5 \times 16^2 + 0 \times 16^1 + 0 \times 16^0 = 5 \times 256 = 1280$

$1280 \div 8 = 160\ (no\ remainder)$

$0x300 = 3 \times 16^2 + 0 \times 16^1 + 0 \times 16^0 = 3 \times 256 = 768$

$768 \div 8 = 256 (no\ remainder)$

Why so ?

As , it depends on $32-$ *bit system and* 64 *system*.

In $64-$ *bit system , memory allocation is total* : 24 *bytes*.

$$\begin{bmatrix} int\ variable - 4\ bytes \\ padding - 4\ bytes \\ Next\ pointer - 8\ bytes \\ Child\ pointer - 8\ bytes \end{bmatrix}$$

Where , as in $32-$ *bit system , memory allocation is total* 12 *bytes.*

Where for next pointer it is allocated -4 *bytes, child pointer it is allocated* 4 *bytes and for integer `data` variable , memory allocated* -4 *bytes.*

Hence for $32-$ *bit system no, such padding is needed.*

Therefore, the structure does not simply double from 12

in $32-$ *bit architecture to* 24 *in* $64-$ *bit architecture.*

It becomes in $64-$ *bit architecture* 24 *because*:

1. *Each pointer grows from* 4 *bytes to* 8 *bytes.*
2. *The compiler inserts* 4 *bytes of padding to satisfy* $8-$ *byte alignment.*

$0x1000$ — *data*
$0x1001$
$0x1002$
$0x1003$

$0x1000$ *is hexadecimal address multiple of* 4 *as it is* 4 *bytes.*

$0x1004$ — *next pointer* ✗

$0x1004$ *is hexadecimal address and not multiple of* 8 *as it is* 8 *bytes.*

$$0x1004 = 1 \times 16^3 + 0 \times 16^2 + 0 \times 16^1 + 4 \times 16^0 = 4096 + 4 = 4100$$

$\frac{4100}{8}$, *has remainder* 4 , *hence not a multiple of* 8.

hence , it adds padding of 4 *bytes.*

$0x1000$ — *data* (4 *bytes*)

$0x1001$

$0x1002$

$0x1003$

$0x1000$ *is* 16 *bit address multiple of* 4 *as it is* 4 *bytes*(*Starting address of data*)

$0x1004$ — *padding* (4*bytes*)

$0x1005$

$0x1006$

$0x1007$

$0x1004$ *is* 16 *bit address multiple of* 4 *as it is* 4 *bytes*(*Starting address of padding*)

$0x1008$ — *next pointer* (8 *bytes*) ✓

$0x1008$ *is* 16 *bit address multiple of* 8 *as it is* 8 *bytes*(*Starting address of padding*)

Alignment applies only to the starting address of an object (***member***).

Address ***Member***

0x1000 → ***data*** (**1*st byte***)
0x1001 → ***data*** (**2*nd byte***)
0x1002 → ***data*** (**3*rd byte***)
0x1003 → ***data*** (**4*th byte***)

0x1000 ***is hexadecimal address multiple of*** **4**
as it is **4** ***bytes***(***Starting address of data***)

0x1004 → ***padding*** (**1*st byte***)
0x1005 → ***padding*** (**2*nd byte***)
0x1006 → ***padding*** (**3*rd byte***)
0x1007 → ***padding*** (**4*th byte***)

0x1004 ***is hexadecimal address multiple of*** **4**
as it is **4** ***bytes***(***Starting address of padding***)

0x1008 → ***next pointer*** (**1*st byte***)
0x1009 → ***next pointer*** (**2*nd byte***)
0x100A → ***next pointer*** (**3*rd byte***)
0x100B → ***next pointer*** (**4*th byte***)
0x100C → ***next pointer*** (**5*th byte***)
0x100D → ***next pointer*** (**6*th byte***)
0x100E → ***next pointer*** (**7*th byte***)
0x100F → ***child pointer*** (**8*th byte***)

0x1008 ***is hexadecimal address multiple of*** **8**
as it is **8** ***bytes***(***Starting address of next***)

0x1010 → ***child pointer*** (**1*st byte***)
0x1011 → ***child pointer*** (**2*nd byte***)
0x1012 → ***child pointer*** (**3*rd byte***)
0x1013 → ***child pointer*** (**4*th byte***)
0x1014 → ***child pointer*** (**5*th byte***)
0x1015 → ***child pointer*** (**6*th byte***)
0x1016 → ***child pointer*** (**7*th byte***)
0x1017 → ***child pointer*** (**8*th byte***)

0x1010 ***is hexadecimal address multiple of*** **8**
as it is **8** ***bytes***(***Starting address of child***)

Now, each addresses holds only 1 byte only aand suppose data value is : 4, it will get stored like:

Decimal : 4

Hex: 0x00000004

$0x1000 \rightarrow data\ (1 \rightarrow byte) =$ 0x04

$0x1001 \rightarrow data\ (1 \rightarrow byte) =$ 0x00

$0x1002 \rightarrow data\ (1 \rightarrow byte) =$ 0x00

$0x1003 \rightarrow data\ (1 \rightarrow byte) =$ 0x00

Now padding :

$0x1004 \rightarrow padding\ (1 \rightarrow byte)$

$0x1005 \rightarrow padding\ (1 \rightarrow byte)$

$0x1006 \rightarrow padding\ (1 \rightarrow byte)$

$0x1007 \rightarrow padding\ (1 \rightarrow byte)$

Next say , 0x3000 is stored at Next pointer as ,

$(0x3000)_{16} = (12288)_{10}$ ***decimal and*** $12288 \div 8 = 1536,$ ***remainder*** $0.$

And $(0x3000)_{16} = (0x0000000000003000)_{16}$

Now, If we split it into bytes (from most significant to least significant), we get:

MSB							*LSB*
00	00	00	00	00	00	30	00

MSB → Most significant byte(1 byte = 8 bits and each bits consists of either 0 or 1) .

LSB → Least significant byte(1 byte = 8 bits and each bits consists of either 0 or 1) .

$0x1008 \rightarrow next\ pointer\ (1 \rightarrow byte) = 0x00$ → *Least Significant Byte*

$0x1009 \rightarrow next\ pointer\ (1 \rightarrow byte) = 0x30$

$0x100A \rightarrow next\ pointer\ (1 \rightarrow byte) = 0x00$

$0x100B \rightarrow next\ pointer\ (1 \rightarrow byte) = 0x00$

$0x100C \rightarrow next\ pointer\ (1 \rightarrow byte) = 0x00$

$0x100D \rightarrow next\ pointer\ (1 \rightarrow byte) = 0x00$

$0x100E \rightarrow next\ pointer\ (1 \rightarrow byte) = 0x00$

$0x100F \rightarrow child\ pointer\ (1 \rightarrow byte) = 0x00$ → *Most Significant Byte*

Next say, $0x5008$ *is stored at child pointer as*,

$(0x5008)_{16} = (20488)_{10}$ *decimal and* $20488 \div 8 = 2561, remainder\ 0$.

And, $(0x5008)_{16} = (0x0000000000005008)_{16}$

Now, If we split it into bytes (from most significant to least significant), we get:

MSB							*LSB*
00	00	00	00	00	00	50	08

MSB → Most significant byte(1 *byte* = 8 *bits and each bits consists of either* 0 *or* 1) .

LSB → Least significant byte(1 *byte* = 8 *bits and each bits consists of either* 0 *or* 1) .

$0x1010 \rightarrow child\ pointer\ (1 \rightarrow byte) = 0x08$ → ***Least Significant Byte***

$0x1011 \rightarrow child\ pointer\ (1 \rightarrow byte) = 0x50$

$0x1012 \rightarrow child\ pointer\ (1 \rightarrow byte) = 0x00$

$0x1013 \rightarrow child\ pointer\ (1 \rightarrow byte) = 0x00$

$0x1014 \rightarrow child\ pointer\ (1 \rightarrow byte) = 0x00$

$0x1015 \rightarrow child\ pointer\ (1 \rightarrow byte) = 0x\ 00$

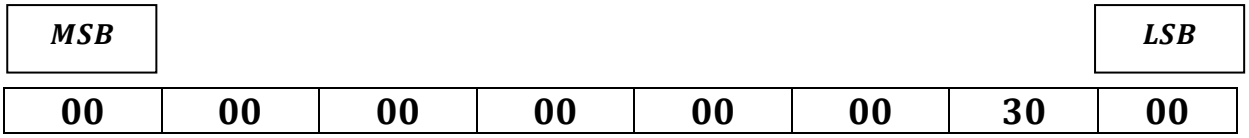
$0x1016 \rightarrow child\ pointer\ (1 \rightarrow byte) = 0x00$

$0x1017 \rightarrow child\ pointer\ (1 \rightarrow byte) = 0x00$ → ***Most Significant Byte***

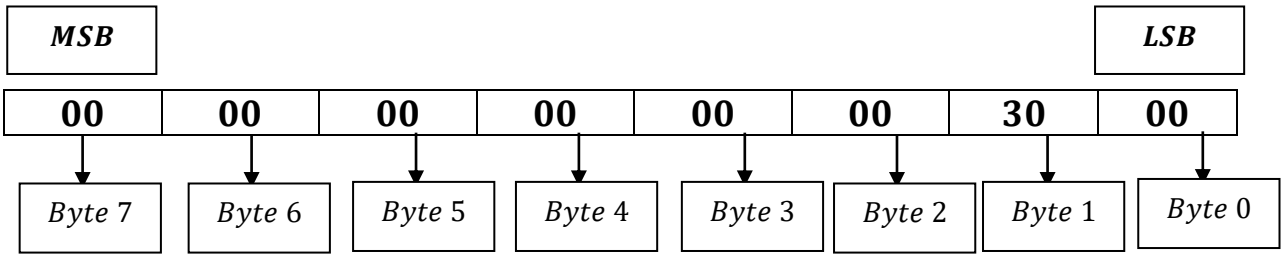
What does this mean ?

$(0x3000)_{16} = (0x0000000000003000)_{16}$

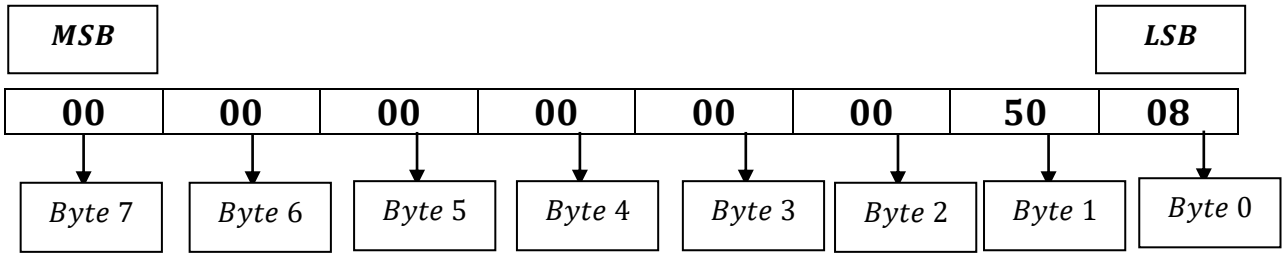
Now, If we split it into bytes (from most significant to least significant), we get:



This actually means:



similarly:



Now , how

***Decimal* : 4**

***Hex*: 0*x*00000004**

0*x*1000 → *data* (1 → *byte*) = 0x04

0*x*1001 → *data* (1 → *byte*) = 0x00

0*x*1002 → *data* (1 → *byte*) = 0x00

0*x*1003 → *data* (1 → *byte*) = 0x00

***is stored*?**

Decimal number $= (4)_{10}$ ***and its binary value is*** $(100)_2$

now int is* 32 *bits* , *so CPU stored it as:

00000000 00000000 00000000 00000100 = 8bits × 4 groups = 32 bits

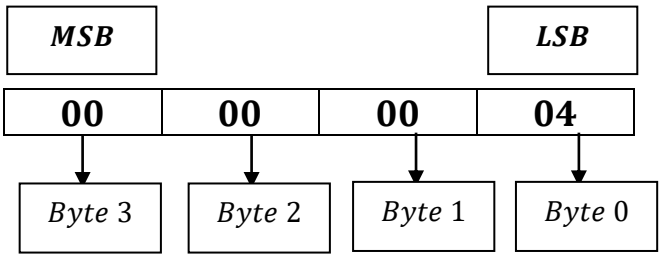
Group every* 4 − *bits:

0000 0000 0000 0000 0000 0000 0000 0100 = 4 bits × 8 groups

becomes:

$(\mathbf{0x00000004})_{16} = \mathbf{7} \rightarrow \mathbf{0}s\ and\ \mathbf{4}.$

Now, If we split it into bytes (from most significant to least significant), we get:



$\mathbf{0}x\mathbf{1000} \rightarrow data\ (\mathbf{1} \rightarrow byte) = \mathbf{0x04}$

$\mathbf{0}x\mathbf{1001} \rightarrow data\ (\mathbf{1} \rightarrow byte) = \mathbf{0x00}$

$\mathbf{0}x\mathbf{1002} \rightarrow data\ (\mathbf{1} \rightarrow byte) = \mathbf{0x00}$

$\mathbf{0}x\mathbf{1003} \rightarrow data\ (\mathbf{1} \rightarrow byte) = \mathbf{0x00}$

Actually everything is stored in Memory address at binary format.

$\mathbf{0}x\mathbf{1000} \rightarrow data\ (\mathbf{1} \rightarrow byte) = \mathbf{0x04}$

$\mathbf{0}x\mathbf{1001} \rightarrow data\ (\mathbf{1} \rightarrow byte) = \mathbf{0x00}$

$\mathbf{0}x\mathbf{1002} \rightarrow data\ (\mathbf{1} \rightarrow byte) = \mathbf{0x00}$

$\mathbf{0}x\mathbf{1003} \rightarrow data\ (\mathbf{1} \rightarrow byte) = \mathbf{0x00}$

As,

$\mathbf{0}x\mathbf{1000} \rightarrow data\ (\mathbf{1} \rightarrow byte) = \mathbf{00000100}\ [\mathbf{8}\ bits]$

$\mathbf{0}x\mathbf{1001} \rightarrow data\ (\mathbf{1} \rightarrow byte) = \mathbf{00000000}[\mathbf{8}\ bits]$

$\mathbf{0}x\mathbf{1002} \rightarrow data\ (\mathbf{1} \rightarrow byte) = \mathbf{00000000}[\mathbf{8}\ bits]$

$\mathbf{0}x\mathbf{1003} \rightarrow data\ (\mathbf{1} \rightarrow byte) = \mathbf{00000000}[\mathbf{8}\ bits]$

Similarly,

The padding bytes are often zero as part of the object's zero initialization or the padding bytes may contain whatever bits happened to be in memory.

Lets initialize with 0s.

$0x1004 \rightarrow padding\ (1 \rightarrow byte) = 0x00$

$0x1005 \rightarrow padding\ (1 \rightarrow byte) = 0x00$

$0x1006 \rightarrow padding\ (1 \rightarrow byte) = 0x00$

$0x1007 \rightarrow padding\ (1 \rightarrow byte) = 0x00$

As,

$0x1004 \rightarrow padding\ (1 \rightarrow byte) = 00000000[8\ bits]$

$0x1005 \rightarrow padding\ (1 \rightarrow byte) = 00000000[8\ bits]$

$0x1006 \rightarrow padding\ (1 \rightarrow byte) = 00000000[8\ bits]$

$0x1007 \rightarrow padding\ (1 \rightarrow byte) = 00000000[8\ bits]$

Similarly,

$0x1008 \rightarrow next\ pointer\ (1 \rightarrow byte) = 0x00$

$0x1009 \rightarrow next\ pointer\ (1 \rightarrow byte) = 0x30$

$0x100A \rightarrow next\ pointer\ (1 \rightarrow byte) = 0x00$

$0x100B \rightarrow next\ pointer\ (1 \rightarrow byte) = 0x00$

$0x100C \rightarrow next\ pointer\ (1 \rightarrow byte) = 0x00$

$0x100D \rightarrow next\ pointer\ (1 \rightarrow byte) = 0x00$

$0x100E \rightarrow next\ pointer\ (1 \rightarrow byte) = 0x00$

$0x100F \rightarrow child\ pointer\ (1 \rightarrow byte) = 0x00$

As,

$0x1008 \rightarrow next\ pointer\ (1 \rightarrow byte) = 00000000[8\ bits]$

$0x1009 \rightarrow next\ pointer\ (1 \rightarrow byte) = 00110000\ [8\ bits]$

$0x100A \rightarrow next\ pointer\ (1 \rightarrow byte) = 00000000[8\ bits]$

$0x100B \rightarrow next\ pointer\ (1 \rightarrow byte) = 00000000[8\ bits]$

$0x100C \rightarrow next\ pointer\ (1 \rightarrow byte) = 00000000[8\ bits]$

$0x100D \rightarrow$ *next pointer* $(1 \rightarrow$ *byte*$) = 00000000[8$ *bits*$]$

$0x100E \rightarrow$ *next pointer* $(1 \rightarrow$ *byte*$) = 00000000[8$ *bits*$]$

$0x100F \rightarrow$ *child pointer* $(1 \rightarrow$ *byte*$) = 00000000[8$ *bits*$]$

Similarly,

$0x1010 \rightarrow$ *child pointer* $(1 \rightarrow$ *byte*$) = 0x08$

$0x1011 \rightarrow$ *child pointer* $(1 \rightarrow$ *byte*$) = 0x50$

$0x1012 \rightarrow$ *child pointer* $(1 \rightarrow$ *byte*$) = 0x00$

$0x1013 \rightarrow$ *child pointer* $(1 \rightarrow$ *byte*$) = 0x00$

$0x1014 \rightarrow$ *child pointer* $(1 \rightarrow$ *byte*$) = 0x00$

$0x1015 \rightarrow$ *child pointer* $(1 \rightarrow$ *byte*$) = 0x\ 00$

$0x1016 \rightarrow$ *child pointer* $(1 \rightarrow$ *byte*$) = 0x00$

$0x1017 \rightarrow$ *child pointer* $(1 \rightarrow$ *byte*$) = 0x00$

As,

$0x1010 \rightarrow$ *child pointer* $(1 \rightarrow$ *byte*$) = 00001000[8$ *bits*$]$

$0x1011 \rightarrow$ *child pointer* $(1 \rightarrow$ *byte*$) = 01010000[8$ *bits*$]$

$0x1012 \rightarrow$ *child pointer* $(1 \rightarrow$ *byte*$) = 00000000[8$ *bits*$]$

$0x1013 \rightarrow$ *child pointer* $(1 \rightarrow$ *byte*$) = 00000000[8$ *bits*$]$

$0x1014 \rightarrow$ *child pointer* $(1 \rightarrow$ *byte*$) = 00000000[8$ *bits*$]$

$0x1015 \rightarrow$ *child pointer* $(1 \rightarrow$ *byte*$) = 00000000[8$ *bits*$]$

$0x1016 \rightarrow$ *child pointer* $(1 \rightarrow$ *byte*$) = 00000000[8$ *bits*$]$

$0x1017 \rightarrow$ *child pointer* $(1 \rightarrow$ *byte*$) = 00000000[8$ *bits*$]$

One small Tweak:

4 *byte address will store* 4 *byte value*[*Multiple of* 4].

8 *byte address will store* 8 *byte value*[*Multiple of* 8].

***it is more accurate to say*:**

An object of size 4 bytes starts at an address that is a multiple of 4.
An object of size 8 bytes starts at an address that is a multiple of 8.

***or, even more precisely*:**

A 4 – byte object is stored starting at an address divisible by 4.

An 8 – byte object is stored starting at an address divisible by 8.

```
int data=4;
```

$0x1000$ ← *Start of 4 – byte object (divisible by 4)* ✓

$0x1001$

$0x1002$

$0x1003$

Similarly,

```
struct Node *next;
```

$0x1008$ ← *Start of 8 – byte object (divisible by 8)* ✓

$0x1009$

$0x100A$

$0x100B$

$0x100C$

$0x100D$

$0x100E$

$0x100F$
